Reflex Habits: Testing, Documentation and Refactoring

Design notes, candidate rule and spitballing

VIDEO GAME CREATION

Purpose of This Document

A running design-notes document, kept separate from the Book Notes & To-Do because it is big enough to need its own space. The Book Notes carry a short entry pointing here. Nothing in this document is a decision yet unless it says so; it holds full ideas, half-formed options and spitballing in one place so they can be sorted later.

Status of the contents: Robert's problem statement and Claude's first-pass commentary are recorded as discussed. Everything under "Spitballing" is deliberately unfiltered.

Robert's Note

In GameMaker in particular, and probably other engines, Robert allowed his own working code to drift from the lesson Keys for personal preferences. That helped flesh out some ideas, but it has become problematic and is slowing him down. He may have to take time and revert his projects to match the Keys.

Refactoring should be part of this course. The groundwork has been laid (Lesson 10's single reset routine, Lesson 11's reused spawn routine) but not taken full advantage of, partly out of the wish to keep the simplest solution and not overwhelm a beginner.

In industry, testing, documentation and refactoring are expected skills but are treated as nice-to-have, and are the first things dropped in a tight schedule. The goal here is to teach them so they become reflex rather than rigor: things done naturally and not seen as separate tasks, done constantly while they are small efforts, instead of waiting for them to build up weight of their own. Open question: can this be integrated suitably into even a beginner-level class, and if so, how?

The Core Idea: Reflex Over Rigor

Rigor is a scheduled event: a testing phase, a documentation sprint, a refactoring week. Events get cut when time is short. Reflex is a small action attached to something students already do, so there is nothing separate to cut. The bet is that a three-minute habit done every week costs less in total, and survives pressure better, than a three-hour task done once the pile has grown.

Test of success: a student never says "now I'm going to test" or "now I'm going to refactor". They say "let me try it" and "that name's wrong" and keep working.

What the Course Already Has

- **Testing:** Every Level 2 is a manual test plan (win then restart, lose then restart, R mid-game does nothing, a full three-wave playthrough). The GDD's Plan to Implement / Implemented / Tested checkboxes already separate "I built it" from "I checked it". Lesson 10's game-state audit and the Stalemate bug hunt are testing lessons presented as features. The Semester 2 Addendum in the Standards already says testing becomes an ongoing habit in Lessons 16 onward.
- **Refactoring:** Lesson 10 pulled setup into one reusable routine (start_game, ResetGame, scr_start_game) with two callers. Lesson 11 reused spawn_enemies from a third caller. Lesson 6's GameManager retrofit and Rule 13 are a refactor too. Rule 6 (Refinement and Style) already calls for a second-look pass. None of these were named as refactors to students.
- **Documentation:** Rules 11 and 15 (meaningful names, beginner-friendly names) are documentation by another route: a name that says what a thing is does the work of a comment. The Complete Script Document planned for the end of the course is documentation of the whole project, and Handouts and Keys are themselves documentation of the intended design.
- **The gap:** All of it is there, but unnamed and unrequired. Students do the activity without ever hearing that it is a skill, and no step in a lab asks them to do it on their own work.

Can It Work at Beginner Level? First-Pass Answer

Yes, if the habits are taught rather than the vocabulary, each habit is small enough to take under five minutes, and each is attached to something that already happens in every lesson. The three habits in beginner terms:

- Try it: run it again, and try to break it, before calling it done. (testing)
- Say it: the name says what it is, and one plain sentence says what the main routine does. (documentation)
- Tidy it: leave the project a little cleaner than you found it; fix the one thing that bothers you now. (refactoring)

One real tension: refactoring needs something to protect it. For beginners the safety net is not unit tests but the manual checks they already have (the Level 2 list). That is why testing has to come first, and why "run it before, run it after" is the key move in any refactor.

The other tension is the course's rule of simplest solution first. The answer proposed here is not to make solutions more complicated, but to make the closing minutes of each lab a fixed, small ritual, so no solution has to carry the habit inside it.

Candidate Rule 16: Definition of Done (DRAFT, not yet adopted)

Draft text for the Standards document, for Robert to accept, rework or reject:

Rule 16: Done Means Tried, Named and Tidy.

Every lab ends the same way, for every student, at every level. A project is done for the week when (1) Try it: the student has run it again and tried at least one case the Handout did not list, (2) Say it: every name they wrote this week says what the thing is, and the week's main routine has one plain sentence saying what it does, and (3) Tidy it: they have fixed or removed one thing that bothers them (a vague name, a dead block, a copy of code that could be a call). The three habits take no more than the last five minutes of lab. They are not a fourth level and are never graded. Handouts carry a one-line "Done means" reminder; the Teacher's Script carries a closing beat; Teacher's Keys note the expected tidy-ups for the week.

How to apply: tie the wording to what already exists (the Level 2 checks are the Try it list; Rules 11 and 15 are the Say it standard; Rule 6's Refinement pass is the Tidy it content). Do not add vocabulary beyond three plain verbs until a student asks. Name a refactor aloud whenever a lesson restructures without changing behavior. Forward-only, like Rule 6; no retrofit of Lessons 1 to 15 is implied.

Concrete Mechanisms (fuller ideas)

- **1. Definition of Done line:** One sentence at the end of each Handout's Level 1 section: Done means you ran it, tried to break it, and left it tidier than you found it. Costs one line of production time per lesson per engine.
- **2. The closing five:** The last five minutes of lab, every week, same shape: three questions on the board. What did you try that the Handout did not tell you to? Is there a name that lies? Is there something you can delete or call instead of copy? Takes a minute each. Share-out at the end can pick one tidy-up from one student.
- **3. Name the refactor when it happens:** When a lesson restructures without changing behavior, say so: This is a refactor: the game plays the same, the code is better shaped. Then show run before, run after. Lessons 6 (GameManager), 10 (reset routine) and 11 (reused spawn routine) are the obvious ones to name in hindsight and in the book.
- **4. Run it before, run it after:** The one safety-net habit. Before any structural change, play the Level 2 checks once; after, play them again. Same behavior is the definition of a refactor.
- **5. Stalemate as the testing lesson:** The Stalemate was only found because some students tried a case the Handout never listed. The book should say that in plain words: this is what testing is, trying the case nobody told you to try.
- **6. The teacher does it on screen:** Every live demo ends with thirty seconds of tidy: rename something, delete a dead block, run it again. Students copy what the instructor does when time is short far more than what the instructor says.
- **7. Sync checkpoint (the drift problem):** After each lesson, compare your own project with the Key and treat every difference as either a deliberate choice or a bug. Drift is not bad in itself (it is how ideas get explored), but un-logged drift makes every later lesson a translation job. See the next section.
- **8. Beginning-of-lesson smoke test:** The first minutes of each lesson already say open your own project from last week. Add: run it, play the Level 2 checks quickly, and confirm it still works before you change it. That is regression testing without the word.

The Drift Problem (Robert's own projects)

Symptom: GameMaker in particular, and probably others, has personal-preference code that no longer matches the Keys, slowing development of each new lesson. Options, from least to most work:

- Revert to the Keys: pick one lesson boundary, rebuild the project from the Key, and from then on treat the Key project as the canonical build. Cleanest, costs a few hours once.
- Keep a variants log: leave the code, but write down every deliberate difference (one line each) so the Keys and the project can be reconciled later. Cheapest now, risk of the log going stale.
- Promote the good drift: where a personal preference is better than the Key, change the Key (and the other engines, per Rule 12) instead of the project. The course gains; this is how the Lesson 10 Scratch and GameMaker corrections happened.

- Make the Keys executable: the Complete Script Document (planned) generated from the same build scripts would give a single, diffable source of truth per engine; a project could be checked against it with a folder compare.

Worth noting in the book: the instructor's own drift is the same problem students will have, at the same scale, which makes it good material for the habits chapter.

Spitballing (half-baked options, in no particular order)

- Test cards: each mechanic gets an index card of three checks (win, lose, R mid-game). Each lesson adds a card. The pile becomes a regression deck run at the start of each lesson. Maybe students write their own card for the week as Level 2.
- Bug Zoo: a class-wide list of named bugs (the Stalemate is the first). Naming a bug makes it searchable, discussable and teachable. Students nominate bugs they found; the best-named one gets added.
- Break it on purpose: two minutes where students swap and try to break a neighbor's game, even across engines (Rule 12 makes cross-engine swaps plausible). This is the Semester 2 informal playtest, started earlier and smaller.
- Explain it to a different-engine student: documentation by speech. Rule 12's common names mean the explanation transfers; this may be the most beginner-friendly form of documentation there is.
- A fourth GDD checkbox: Plan to Implement / Implemented / Tested / Tidied. Probably too much; maybe replace nothing and use only the first three, since adding a state invites box-ticking.
- Rule of three, plain version: if you wrote it twice, give it a name; if three times, it needs to be a function. Lesson 10 and 11 already teach this implicitly.
- Dead-code funeral: celebrate deleting. A share-out line: who deleted something this week? Less code is a win, and students rarely believe that until someone says it.
- Comments that say why, not what: one plain-language why per routine. Possibly introduced only in Semester 2; Semester 1 relies on names.
- A refactor-only lesson in Semester 2: the game looks and plays the same at the end, and every student can show what is better. A whole lesson where nothing new appears would make the idea unmissable.
- Save a dated copy as the beginner version of version control: Lesson 11 start, Lesson 11 done. Real git later, perhaps; it may be too heavy for middle school.
- A tidy tax: ten percent of lab time is reserved, unavoidably, rather than depending on students having time left. Probably the only way it survives a crowded lab.
- Make the Level 3 menu always contain one Refine item (Rule 6 already says this). Alternatively promote one Refine item per lesson into Level 2 so everyone does it.
- Use AI critique as the tidy step (fits the instructor's own AI and agentic coding thread): students paste a routine and ask what is confusing about its names, then decide for themselves. Needs care: the habit has to stay the student's, not the tool's. Maybe Semester 2 only.
- Tell students the industry truth: these three things get dropped first when time is tight, and the people who do them anyway are the ones teams want to keep. Honest framing may motivate older students more than a rule does.
- Handout margin icons: a small try / name / tidy icon beside the step where the habit fits, so the habit is visible in the flow of the lesson rather than collected at the end.

- Peer rotation: each week one student is the tester for another; the tester writes three observations, one must be good. Cross-engine pairs where possible.
- Lab Time Notes already carry a game-state-drift item; add a standing question the instructor asks while circulating: Show me something you tried that wasn't in the Handout.

Risks and Things to Watch

- Checkbox culture: habits that become a ritual box with no thought behind them. Mitigation: keep the questions open-ended and share one answer aloud, rather than collect a tick.
- Overwhelm: adding anything to a beginner's lab. Mitigation: it is the last five minutes, not a new task, and the Handout never lists more than one line about it.
- Lab time: the closing five already competes with the exit check and share-out. The agenda may need to move five minutes.
- Engine unevenness: refactoring is easy in script engines, harder in block engines (Scratch's end blocks, GameMaker Visual nesting). The tidy examples need to be engine-appropriate, per Rule 5 and Rule 7.
- Pace: the course is already full. Rule 16 as drafted adds nothing that needs a new lesson; any idea that would is Semester 2 or book-only.

Connections to Existing Rules

- Rule 6 (Refinement and Style) is the Tidy it content; Rule 16 would give it a fixed place in the lab rather than only in Level 3.
- Rules 11 and 15 (names) are the Say it standard.
- Rule 12 (cross-engine consistency) makes cross-engine peer explanation and testing possible.
- Rule 13 (GameManager owns shared state) is a refactor students already did once; the habit names it.
- Rule 14 (code completeness) makes the Keys a reference for sync checkpoints.
- Semester 2 Addendum (testing becomes ongoing) is where Try it grows from informal to structured.
- Complete Script Document (planned): the natural source of truth for drift checks.

Decisions Needed From Robert

- Adopt, rework or drop the draft Rule 16?
- Which of the closing-five questions are right for middle school versus high school?
- Does the closing five replace part of the exit check, or does the agenda get five more minutes?
- For the GameMaker drift: revert, log, promote, or a mix? Which lesson is the cleanest boundary to revert to?
- Start the habits at Lesson 12 (forward-only), or fold a first version into the Lesson 11 review as a pilot?
- Should any spitball item (test cards, Bug Zoo, refactor-only lesson) be promoted to a to-do?